



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 10. Работа с файлами

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Работа с файлами в файловой системе
-  Чтение и запись файлов
-  Сериализация данных
-  Формат JSON
-  Формат XML
-  Конфигурационные файлы
-  Формат TOML
-  Формат YAML

Поиск файлов

Модуль glob

- ⦿ Находит файлы и директории, удовлетворяющие заданным шаблонам путей (поддерживаются '*' и '?')

Функция `glob.glob()` возвращает список имен файлов и директорий, удовлетворяющий заданному шаблону

```
glob.glob(pathname, *, root_dir=None, dir_fd=None,  
          recursive=False, include_hidden=False)
```

- ⦿ `root_dir` – директория, где выполняется поиск (**None** – текущая)
- ⦿ `recursive=True` – поиск во вложенных директориях ('**' обозначает любой уровень вложения)
- ⦿ `include_hidden=True` – включать в результаты поиска скрытые файлы и директории

Пример поиска файлов

```
├── 1.gif
├── 2.txt
├── card.gif
└── sub
    └── 3.txt
```

```
import glob
glob.glob('./[0-9].*')
# ['./1.gif', './2.txt']
glob.glob('*.gif')
# ['1.gif', 'card.gif']
glob.glob('?.gif')
# ['1.gif']
glob.glob('**/*.txt', recursive=True)
# ['2.txt', 'sub/3.txt']
glob.glob('./**/', recursive=True)
# ['./', './sub/']
```

Операции с файлами (1/3)

Модуль `shutil`

- ⦿ Операции с файлами и группами файлов: копирование, перемещение и удаление

 `shutil.copy()` копирует файл, сохраняя его настройки безопасности, но не сохраняя время создания/изменения

```
shutil.copy(src, dst, *, follow_symlinks=True)
```

 `shutil.copy2()` копирует файл, сохраняя его настройки безопасности и время создания/изменения

```
shutil.copy2(src, dst, *, follow_symlinks=True)
```

Операции с файлами (2/3)

 `shutil.move()` перемещает файл или директорию*

```
shutil.move(src, dst, copy_function=copy2)
```

 `shutil.copytree()` рекурсивно копирует вложенные файлы и директории*

```
shutil.copytree(src, dst, symlinks=False, ignore=None,  
                copy_function=copy2, ignore_dangling_symlinks=False,  
                dirs_exist_ok=False)
```

* - имеют режимы, соответствующие `copy` и `copy2`

Операции с файлами (3/3)

- 🌀 `shutil.rmtree()` рекурсивно копирует вложенные файлы и директории

```
shutil.rmtree(path, ignore_errors=False, onerror=None, *,  
              onexc=None, dir_fd=None)
```

- 🌀 `shutil.chown()` меняет владельца и/или группу

```
shutil.chown(path, user=None, group=None, *, dir_fd=None,  
             follow_symlinks=True)
```

Операции с файлами (3/3)

- 🌀 `shutil.rmtree()` рекурсивно копирует вложенные файлы и директории

```
shutil.rmtree(path, ignore_errors=False, onerror=None, *,  
              onexc=None, dir_fd=None)
```

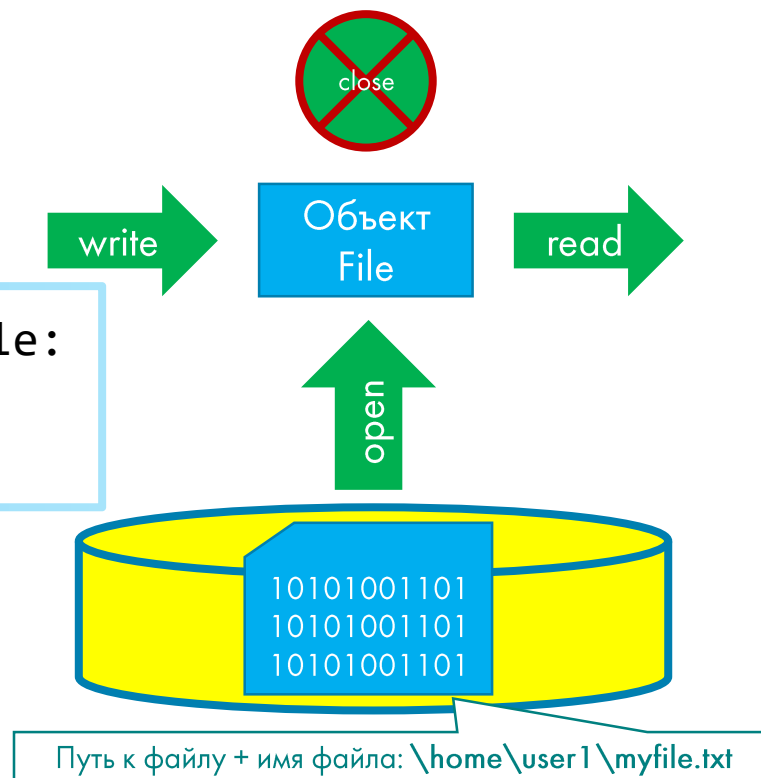
- 🌀 `shutil.chown()` меняет владельца и/или группу

```
shutil.chown(path, user=None, group=None, *, dir_fd=None,  
             follow_symlinks=True)
```

Объект File в Python

- Функция `open()` открывает файл и возвращает объект, позволяющий читать и/или записывать данные
- Выражение `with` гарантирует, что после использования файл будет закрыт

```
with open(file_name, ...) as work_file:  
    work_file.read(...)  
    work_file.write(...)
```



Открытие файла

Функция `open()`

- Если кодировка текста не задана, будет использоваться кодировка операционной системы
 - MS Windows: `cp1251`
 - Linux: `utf-8`

```
file = open(file_name, mode = 'rt', encoding = 'utf-8')
```

Символ	Режим, в котором открывается файл
'r'	Чтение (по умолчанию)
'w'	Запись с затиранием
'x'	Создание файла (ошибка если файл существует)
'a'	Запись с добавлением в конец
'b'	Бинарный режим
't'	Текстовый режим (по умолчанию)
'+'	Чтение и запись без затирания

Чтение файла

```
with open('myfile.txt') as mf:
    str = mf.read()
    # 'Line 1 text\nLine 2 text'

    str = mf.read(15)
    # 'Line 1 text\nLin'

    str = mf.readline()
    # 'Line 1 text\n'

    str = mf.readline(15)
    # 'Line 1 text\n'

    str = mf.readline(6)
    # 'Line 1'
```

myfile.txt

Line 1 text
Line 2 text

Запись файла

```
with open('myfile.txt', 'w') as mf:  
    n = mf.write('New text')  
    # n = 8
```

myfile.txt

New text

```
with open('myfile.txt', 'a') as mf:  
    mf.write('NEW')  
    # n = 3
```

myfile.txt

Line 1 text
Line 2 textNEW

Сериализация

Модуль pickle

- ⦿ Сериализация – представление структуры и состояния объекта Python в виде последовательности байт (запись в файл)
- ⦿ Десериализация – восстановление ранее сериализованного объекта (чтение из файла)

 `pickle.dump()` записывает объект в бинарный файл

```
pickle.dump(obj, file)
```

 `pickle.load()` восстанавливает объект из файла

- ⦿ Возвращает объект

```
pickle.load(file)
```

 Десериализация объектов из файлов, полученных из непроверенных источников ОПАСНА

Формат JSON

Модуль json

- ⦿ Представление структур данных Python в виде текста в формате JSON
- ⦿ Создание структур данных Python на основе JSON-строк

 `json.dump()` записывает объект в текстовый файл

 `json.dumps()` возвращает JSON-строку

 `json.load()` создает объект из JSON-файла

- ⦿ Возвращает объект

 `json.loads()` создает объект из JSON-строки

- ⦿ Возвращает объект

 Загрузка объектов из файлов, полученных из непроверенных источников ОПАСНА

Преобразование типов

Python	JSON
dict	object
list, tuple	array
str	string
int, float, int- & float-derived Enums	number
True	true
False	false
None	null

```
{ "elements":  
  [ { "symbol": "H",  
        "number": 1,  
        "weight": 1 },  
    { "symbol": "C",  
        "number": 6,  
        "weight": 12 },  
    { "symbol": "O",  
        "number": 8,  
        "weight": 16 } ] }
```

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	float
true	True
false	False
null	None

Примеры: запись (1/2)

```
import json

json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
# '["foo", {"bar": ["baz", null, 1.0, 2]}]'

print(json.dumps("\foo\bar"))
# "\"foo\bar\""

print(json.dumps('\u1234'))
# "\"u1234\""

print(json.dumps('\\\\'))
# "\"\\\""

print(json.dumps({"c": 0, "b": 0, "a": 0}, sort_keys=True))
# {"a": 0, "b": 0, "c": 0}

with open('myfile.json') as file:
    json.dump(['To file'], file)
```

Примеры : запись (2/2)

Компактное представление

```
json.dumps([1, 2, 3, {'4': 5, '6': 7}], separators=(',', ':'))  
# '[1,2,3,{"4":5,"6":7}]'
```

Форматированный вывод

```
print(json.dumps({'6': 7, '4': 5}, sort_keys=True, indent=4))  
# {  
#     "4": 5,  
#     "6": 7  
# }
```

Примеры: чтение

```
import json

json.loads('["foo", {"bar":["baz", null, 1.0, 2]}]')
#['foo', {'bar': ['baz', None, 1.0, 2]}]

json.loads('"\\\\"foo\\\\"bar"')
#"foo\x08ar"

with open('myfile.json') as file:
    json.load(file)
# ['To file']
```

Формат XML

Модуль `xml.dom`

- ⦿ Элементы XML документа загружаются в память в виде иерархии объектов
- ⦿ Реализует Document Object Model API
 - Перемещение по дереву объектов
 - Чтение/запись данных объектов (значений элементов и атрибутов)
 - Преобразование структуры дерева
 - Поиск элементов по имени, значению, атрибуту, положению

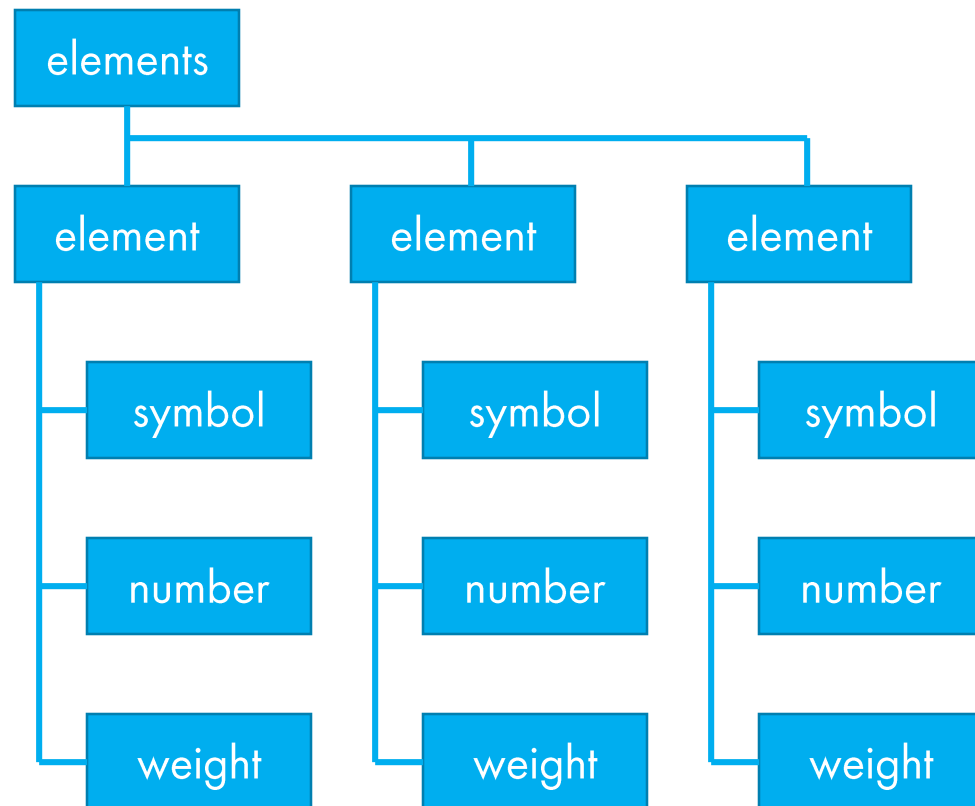
Модуль `xml.sax`

- ⦿ Поточная обработка документа
 - Теги обрабатываются как события
- ⦿ Simple API for XML
 - Приложение должно само обрабатывать события (интерпретировать теги)

Загрузка объектов из файлов, полученных из непроверенных источников ОПАСНА

Пример: DOM

```
<!-- XML -->
<elements>
  <element>
    <symbol>H</symbol>
    <number>1</number>
    <weight>1</weight>
  </element>
  <element>
    <symbol>C</symbol>
    <number>6</number>
    <weight>12</weight>
  </element>
  <element>
    <symbol>O</symbol>
    <number>8</number>
    <weight>16</weight>
  </element>
</elements>
```



Конфигурационные файлы

Модуль configparser

- Чтение и запись информации из конфигурационных файлов (INI)

```
[DEFAULT]
ServerAliveInterval = 45
Compression = yes
CompressionLevel = 9
ForwardX11 = yes

[forge.example]
User = hg

[topsecret.server.example]
Port = 50022
ForwardX11 = no
```

Пример: чтение (1/2)

```
config = configparser.ConfigParser()

config.sections()
# []
config.read('example.ini')
# ['example.ini']
config.sections()
# ['forge.example', 'topsecret.server.example']
'forge.example' in config
# True
'python.org' in config
# False
config['forge.example']['User']
# 'hg'
config['DEFAULT']['Compression']
# 'yes'
```

Пример: чтение (2/2)

```
topsecret = config['topsecret.server.example']
topsecret['ForwardX11']
# 'no'

opsecret['Port']
# '50022'

for key in config['forge.example']:
    print(key)
# user
# compressionlevel
# serveraliveinterval
# compression
# forwardx11

config['forge.example']['ForwardX11']
# 'yes'
```

Пример: запись

```
import configparser

config = configparser.ConfigParser()

config['DEFAULT'] = {'ServerAliveInterval': '45',
                    'Compression': 'yes',
                    'CompressionLevel': '9'}

config['forge.example'] = {}
config['forge.example']['User'] = 'hg'
config['topsecret.server.example'] = {}
topsecret = config['topsecret.server.example']
topsecret['Port'] = '50022'      # mutates the parser
topsecret['ForwardX11'] = 'no'  # same here
config['DEFAULT']['ForwardX11'] = 'yes'

with open('example.ini', 'w') as configfile:
    config.write(configfile)
```

Формат TOML

Модуль tomllib

- ◉ Чтение данных из TOML-файлов
- ◉ TOML (Tom's Obvious Minimal Language) — это формат конфигурационных файлов, похожий на INI, но более мощный и удобный
 - Он поддерживает типы данных, вложенные структуры и часто используется в современных Python-проектах

```
name = "Алиса"
age = 30
skills = ["Python", "Анализ данных", "SQL"]

[address]
city = "Москва"
street = "Тверская, 42"
postal_code = "123456"
```

Пример: чтение

```
import tomllib

with open("config.toml", "rb") as f:  # Тоже бинарный режим!
    config = tomllib.load(f)

print(config)

# {
#   "name": "Алиса",
#   "age": 30,
#   "skills": ["Python", "Анализ данных", "SQL"],
#   "address": {
#     "city": "Москва",
#     "street": "Тверская, 42",
#     "postal_code": "123456"
#   }
# }
```

Преобразование типов

TOML	Python
TOML document	dict
string	str
integer	int
float	float (configurable with parse_float)
boolean	bool
offset date-time	datetime.datetime (tzinfo attribute set to an instance of datetime.timezone)
local date-time	datetime.datetime (tzinfo attribute set to None)
local date	datetime.date
local time	datetime.time
array	list
table	dict
inline table	dict
array of tables	list of dicts

Формат YAML

Модуль PyYAML (требуется установка)

- Чтение и запись данных в формате YAML
- YAML (YAML Ain't Markup Language) — это формат для хранения и обмена данными, похожий на JSON, но более удобный для восприятия человеком

```
name: Алиса
age: 30
skills:
  - Python
  - Анализ данных
  - SQL
address:
  city: Москва
  street: Тверская, 42
  postal_code: "123456"
```

```
pip install pyyaml
```

Пример: запись

```
import yaml

data = {
    "name": "Алиса",
    "age": 30,
    "skills": ["Python", "Анализ данных", "SQL"],
    "address": {
        "city": "Москва",
        "street": "Ленина, 42",
        "postal_code": "123456"
    }
}

# Запись в файл
with open("config.yaml", "w") as file:
    yaml.dump(data, file, default_flow_style=False)
```

Пример: чтение

```
import yaml

with open("config.yaml", "r") as file:
    loaded_data = yaml.safe_load(file)

print(loaded_data)

# {
#     "name": "Алиса",
#     "age": 30,
#     "skills": ["Python", "Анализ данных", "SQL"],
#     "address": {
#         "city": "Москва",
#         "street": "Тверская, 42",
#         "postal_code": "123456"
#     }
# }
```

Сравнение форматов

- 🌐 TOML — конфигурации (например, настройки проекта)
- 🌐 YAML — сложные конфиги (Docker, Kubernetes), где важны комментарии
- 🌐 JSON — API и обмен данными между сервисами

Особенность	TOML	JSON	YAML
Читаемость	+++ (как INI)	+ (строгий синтаксис)	+++ (отступы)
Поддержка типов	Да	Да	Да
Комментарии	# Да	Нет	# Да
Вложенность	Через [секции]	Через { }	Через отступы
Использование	Конфиги (например, pyproject.toml)	API, Web	Конфиги, DevOps

Итоги

В данной лекции были рассмотрены следующие темы:

-  Работа с файлами в файловой системе
-  Чтение и запись файлов
-  Сериализация данных
-  Формат JSON
-  Формат XML
-  Конфигурационные файлы
-  Формат TOML
-  Формат YAML